

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 5: Formularios y Consumo de APIs

Introducción al Módulo

En el desarrollo de software empresarial contemporáneo, los formularios interactivos y el consumo de servicios REST constituyen la columna vertebral de cualquier cliente web moderno. A lo largo de este módulo adquirirás las destrezas técnicas necesarias para estructurar formularios tipados y declarativos mediante Formularios Reactivos, definir validaciones síncronas en tiempo real con reglas nativas y personalizadas, orquestar peticiones HTTP mediante el cliente funcional con `provideHttpClient()` e integrar mecanismos preventivos ante anomalías de red aplicando operadores avanzados de RxJS. Complementarás tu asimilación técnica consultando el glosario conceptual (ver ANEXO I), el manual de códigos de estado HTTP (ver ANEXO II) y la guía de interoperabilidad entre Observables y Signals (ver ANEXO III).

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Introducción y Objetivos del Módulo:** Alcance técnico, competencias formativas de ingeniería y criterios de éxito operacional.
- **2 Formularios Reactivos en Angular:** Fundamentos del modelo declarativo, generación mediante Angular CLI, clases atómicas (`FormControl`, `FormGroup`) y vinculación bidireccional limpia con plantillas.
- **3 Validaciones Estrictas: Básicas y Avanzadas:** Reglas estándar con `Validators`, sincronización visual y validadores personalizados para el dominio empresarial.
- **4 El Cliente Angular HttpClient:** Inyección funcional con `provideHttpClient()`, scaffolding de servicios, contratos fuertemente tipados con TypeScript y arquitectura de operaciones CRUD.
- **5 Manejo de Errores Robustos con RxJS:** Resiliencia de red, operador `catchError`, políticas de reintento con `retry` y normalización de excepciones en cliente.
- **6 Ejercicio Final:** Integración integral de formulario de alta validado en tiempo real consumiendo una API Mock con base de datos simulada y persistencia local.
- **7 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de competencias, directrices de arquitectura y patrones asimilados en la unidad.

Anexos Técnicos de Referencia:

- **ANEXO I (Glosario de Términos Técnicos):** Compendio conceptual con la definición rigurosa de términos clave citados con (ver ANEXO I).

- **ANEXO II (Manual de Códigos de Estado HTTP en APIs REST):** Guía sistemática de códigos de respuesta estructurados por familias (2xx, 3xx, 4xx y 5xx), citado con (ver ANEXO II).
- **ANEXO III (Observables en Angular v19):** Arquitectura reactiva basada en flujos, funciones de interoperabilidad (`toSignal`, `toObservable`) y prevención de fugas de memoria, citado con (ver ANEXO III).

1 Introducción y Objetivos del Módulo

El presente módulo te proporciona los criterios arquitectónicos y las habilidades prácticas indispensables para concebir, maquetar e integrar flujos de captura de datos y comunicación asíncrona de nivel corporativo en Angular 19.

En el desarrollo empresarial, los formularios web no son simples agrupaciones de campos de texto, sino compuertas lógicas que deben garantizar la integridad estructural de la información antes de transferirla a la red. Paralelamente, el consumo de servicios RESTful exige diseñar capas de datos altamente resilientes, capaces de comunicarse asíncronamente con servidores remotos, interpretar con precisión la semántica de las respuestas HTTP y recuperarse de manera preventiva de incidencias o latencias no deseadas.

Al culminar satisfactoriamente este módulo formativo, serás capaz de:

- Modelar formularios complejos de forma declarativa, reactiva e inmutable mediante las clases nativas de Angular.
- Implementar mecanismos estrictos de validación síncrona en tiempo real, evaluando estados de interacción (`pristine`, `dirty`, `touched`, `valid`) para enriquecer la experiencia de usuario (UX) (ver ANEXO I).
- Gestionar la transferencia de datos asíncrona mediante el cliente `HttpClient` configurado funcionalmente en el arranque de la aplicación.
- Controlar anomalías e interrupciones de red integrando operadores de RxJS como `catchError` y `retry` dentro de tuberías declarativas (ver ANEXO III).
- Aprovisionar y conectar una arquitectura cliente-servidor simulada localmente (API Mock) con JSON Server para pruebas de ciclo completo (ver ANEXO I).

2 Formularios Reactivos en Angular

La aproximación reactiva de Angular a la gestión de datos de entrada se fundamenta en un flujo de datos explícito, inmutable y totalmente testeable. A diferencia del enfoque guiado por plantillas (*Template-driven Forms*), en los formularios reactivos la lógica y la estructura del árbol de controles se definen directamente en la clase TypeScript del componente, mapeándose luego de forma unívoca a los elementos visuales del DOM.

Este paradigma aporta ventajas decisivas en aplicaciones a gran escala:

- **Predictibilidad y Tipado Estricto:** Puedes inferir o declarar contratos TypeScript rigurosos para cada campo o grupo del formulario.

- **Inmutabilidad y Trazabilidad:** Cada modificación de valor genera un evento observable que permite auditoría y monitoreo continuo sin depender de `Zone.js`.
- **Facilidad de Pruebas Unitarias:** Al no depender de la renderización del navegador para existir, puedes instanciar y probar la lógica de validación de un formulario sin necesidad de compilar la plantilla HTML.

2.1 Generación del Componente con Angular CLI

Para comenzar a trabajar con formularios reactivos en una aplicación moderna basada en componentes autónomos (Standalone, ver ANEXO I), abres la terminal integrada en la raíz del proyecto y ejecutas el comando oficial de generación de componentes:

```
ng generate component components/registro --standalone
# Comando abreviado equivalente:
ng g c components/registro
```

Al ejecutar esta instrucción, Angular CLI realiza de manera automatizada las siguientes operaciones en el disco:

- Crea la carpeta `src/app/components/registro/`.
- Genera el archivo TypeScript `registro.component.ts` configurado con `standalone: true`.
- Genera la plantilla HTML `registro.component.html` para la vista del formulario.
- Genera la hoja de estilos `registro.component.css` para el aislamiento visual del componente.
- Genera el archivo de pruebas unitarias `registro.component.spec.ts`.

2.2 El Pilar Fundamental: FormControl

El objeto `FormControl` representa la unidad mínima de información y control en un formulario de Angular. Permite realizar el seguimiento exhaustivo del valor de una entrada interactiva (campo de texto, selector, casilla de verificación), administrando individualmente propiedades críticas:

- **Valor actual** (`value`): Dato primitivo u objeto almacenado en el control.
- **Estado de validez** (`valid` / `invalid`): Determina si el valor actual satisface las reglas de validación asignadas.
- **Historial de modificación** (`pristine` / `dirty`): `pristine` indica que el valor inicial no ha cambiado; `dirty` confirma que has alterado manualmente el campo (ver ANEXO I).
- **Interacción visual** (`untouched` / `touched`): `touched` se activa en el instante en que el control pierde el foco visual (*blur*) (ver ANEXO I).

2.3 Agrupamiento y Estructura: FormGroup

La clase `FormGroup` actúa como un contenedor lógico que agrupa varias instancias de `FormControl` u otros `FormGroup` anidados. Permite gestionar el estado global del formulario de forma centralizada: si un solo control dentro del grupo es inválido, todo el grupo hereda automáticamente el estado de invalidez (`invalid`). Esta propagación simplifica tareas operativas como condicionar la habilitación del botón de envío a la corrección absoluta de todos los campos.

2.4 Implementación Técnica de Referencia

A continuación, observa la implementación de un formulario reactivo en un componente autónomo (Standalone, ver ANEXO I) mediante la importación directa de `ReactiveFormsModule`. Editamos el archivo `src/app/components/registro/registro.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { FormGroup, FormControl, Validators, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-registro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './registro.component.html'
})
export class RegistroComponent implements OnInit {
  miFormulario!: FormGroup;

  ngOnInit(): void {
    this.miFormulario = new FormGroup({
      nombre: new FormControl('', Validators.required),
      correo: new FormControl('', [Validators.required, Validators.email])
    });
  }

  onSubmit(): void {
    if (this.miFormulario.valid) {
      console.log('Datos procesados:', this.miFormulario.value);
    }
  }
}
```

Listing 1: p2-formularios/registro.component.ts

La plantilla HTML se enlaza al modelo lógico mediante las directivas estructurales `[formGroup]` y `formControlName` dentro de `src/app/components/registro/registro.component.html`:

```
<form [formGroup]="miFormulario" (ngSubmit)="onSubmit()">
  <div>
    <label for="nombre">Nombre Completo: </label>
    <input id="nombre" type="text" formControlName="nombre">
  </div>
  <div>
    <label for="correo">Correo Electrónico: </label>
    <input id="correo" type="email" formControlName="correo">
  </div>
  <button type="submit" [disabled]="miFormulario.invalid">Registrar</button>
</form>
```

Listing 2: p2-formularios/registro.component.html

Ejercicio: Formulario Reactivo de Autenticación Rápida

Enunciado Genera mediante Angular CLI un componente standalone denominado

LoginRapidoComponent ubicado en `src/app/components/login-rapido/`. El componente debe configurar un `FormGroup` con dos campos: `usuario` (obligatorio) y `password` (obligatorio con longitud mínima de 6 caracteres). La vista debe mantener deshabilitado el botón de acceso mientras el formulario sea inválido, exhibir advertencias contextuales al perder el foco (`touched`) e imprimir en consola el objeto de credenciales únicamente tras una validación exitosa.

Solución Ejecuta en tu terminal el comando de andamiaje:

```
ng g c components/login-rapido --inline-template --inline-style
```

Implementa el componente completo en `src/app/components/login-rapido/login-rapido.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { FormGroup, FormControl, Validators, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-login-rapido',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <div class="login-container">
      <form [formGroup]="loginForm" (ngSubmit)="iniciarSesion()" class="login-card">
        <h3>Acceso al Sistema</h3>
        <div class="form-field">
          <label for="usuario">Identificador de Usuario: </label>
          <input id="usuario" type="text" formControlName="usuario" placeholder="Ej. admin_01"
        ↵ />
          @if (loginForm.get('usuario')?.invalid && loginForm.get('usuario')?.touched) {
            <small class="error-text">El identificador de usuario es obligatorio.</small>
          }
        </div>
        <div class="form-field">
          <label for="password">Clave Secreta: </label>
          <input id="password" type="password" formControlName="password" placeholder="Mínimo 6
        ↵ caracteres" />
          @if (loginForm.get('password')?.invalid && loginForm.get('password')?.touched) {
            <small class="error-text">La clave debe contener al menos 6 caracteres.</small>
          }
        </div>
        <button type="submit" [disabled]="loginForm.invalid">Acceder al Panel</button>
      </form>
    </div>
  `,
  styles: [`
    .login-container { max-width: 360px; margin: 10px auto; font-family: 'Noto Sans', sans-serif
    ↵ ; }
    .login-card { background: #FFFFFF; border: 1px solid #CBD5E1; padding: 18px; border-radius:
    ↵ 8px; }
    h3 { color: #1F2758; margin-top: 0; }
    .form-field { margin-bottom: 12px; display: flex; flex-direction: column; gap: 4px; font-
    ↵ size: 12px; }
    input { padding: 6px 10px; border: 1px solid #CBD5E1; border-radius: 4px; }
    .error-text { color: #DC2626; font-size: 11px; font-weight: bold; }
    button { background: #1E75B8; color: #fff; border: none; padding: 8px 14px; border-radius: 4
  `],
})
export class LoginRapidoComponent implements OnInit {
  loginForm: FormGroup;

  ngOnInit(): void {
    this.loginForm = new FormGroup({
      usuario: new FormControl('', { validators: [Validators.required] }),
      password: new FormControl('', { validators: [Validators.required, Validators.minLength(6)] })
    });
  }

  iniciarSesion(): void {
    console.log(this.loginForm.value);
  }
}
```

```
↩ px; cursor: pointer; font-weight: bold; width: 100%; }
button:disabled { background: #CBD5E1; cursor: not-allowed; }
`]
})
export class LoginRapidoComponent implements OnInit {
  loginForm!: FormGroup;

  ngOnInit(): void {
    this.loginForm = new FormGroup({
      usuario: new FormControl('', Validators.required),
      password: new FormControl('', [Validators.required, Validators.minLength(6)])
    });
  }

  iniciarSesion(): void {
    if (this.loginForm.valid) {
      console.log('Credenciales verificadas y listas para emisión:', this.loginForm.value);
    }
  }
}
```

Listing 3: p2-ej/login-rapido.component.ts

Explicación

- **Estructuración Declarativa en OnInit:** Al instanciar `FormGroup` en el gancho `ngOnInit`, aíslas la configuración lógica del ciclo de renderizado del navegador, permitiendo inspeccionar el estado antes del primer pintado.
- **Bloqueo Preventivo del DOM:** La vinculación `[disabled]="loginForm.invalid"` asegura que el usuario no pueda emitir peticiones incompletas, garantizando interfaces con retroalimentación instantánea.
- **Evaluación de Foco con Touched:** Al condicionar los mensajes de error a la propiedad `touched`, previenes que aparezcan advertencias prematuras en rojo antes de que el usuario haya interactuado con el campo.

3 Validaciones Estrictas: Básicas y Avanzadas

Las validaciones garantizan la integridad, formato y coherencia de los datos antes de su transferencia hacia cualquier API de persistencia. Angular provee un catálogo de validadores síncronos nativos y habilita la definición de reglas de negocio altamente personalizadas.

3.1 Validaciones Básicas y Nativas

A través de la clase estática `Validators`, accedes de forma directa a las siguientes comprobaciones comunes:

- `Validators.required`: Comprueba que el control contenga un valor no nulo ni una cadena en blanco.
- `Validators.minLength(n)` y `Validators.maxLength(n)`: Acotan la longitud permitida de caracteres.

- `Validators.email`: Verifica que el contenido satisfaga la estructura formal de una dirección de correo electrónico estándar.
- `Validators.pattern(regex)`: Evalúa el contenido del control frente a una expresión regular personalizada.

3.2 Validaciones Avanzadas (Personalizadas)

Cuando las directivas predeterminadas resultan insuficientes para las normas de tu organización, se definen funciones validadoras personalizadas (*Custom Validators*). Una función validadora recibe una referencia abstracta (`AbstractControl`) y retorna un objeto de errores (`ValidationErrors`) si la validación falla, o estrictamente `null` si el valor cumple la regla.

Para mantener una arquitectura limpia y ordenada, crea un archivo dedicado en `src/app/validators/validadores-especiales.ts`:

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

export class ValidadoresEspeciales {
  static esDominioEmpresa(control: AbstractControl): ValidationErrors | null {
    const email = control.value as string;
    if (email && !email.endsWith('@itformacion.com')) {
      return { dominioInvalido: true };
    }
    return null;
  }
}
```

Listing 4: p3-validaciones/validadores-especiales.ts

Ejercicio: Validador de Rango Numérico para Puertos de Red

Enunciado Diseña un validador personalizado denominado `puertoValido` que verifique que el número ingresado en un control pertenezca al rango de puertos TCP válidos para servicios de usuario (entre 1024 y 65535). Demuestra su aplicación práctica en un componente standalone completo (`ConfiguracionRedComponent`) creado con Angular CLI en `src/app/components/config-red/`, exponiendo el mapa descriptivo del error en caso de fallo.

Solución Genera el componente ejecutando:

```
ng g c components/config-red --inline-template --inline-style
```

Implementa el validador y el componente en `src/app/components/config-red/config-red.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { AbstractControl, ValidationErrors, FormControl, FormGroup, Validators,
  ReactiveFormsModule } from '@angular/forms';

export function puertoValido(control: AbstractControl): ValidationErrors | null {
  const valor = Number(control.value);
```

```

    if (control.value !== null && control.value !== '' && (isNaN(valor) || valor < 1024 || valor >
        ↪ 65535)) {
        return {
            puertoFueraDeRango: {
                valorActual: control.value,
                minimoPermitido: 1024,
                maximoPermitido: 65535
            }
        };
    }
    return null;
}

@Component({
    selector: 'app-config-red',
    standalone: true,
    imports: [ReactiveFormsModule],
    template: `
        <div class="config-box">
            <h4>Parámetros de Conectividad TCP</h4>
            <form [formGroup]="formRed" (ngSubmit)="guardarParametros()">
                <div class="campo">
                    <label for="puerto">Puerto TCP de Servicio:</label>
                    <input id="puerto" type="number" formControlName="puertoServicio" />
                    @if (formRed.get('puertoServicio')?.hasError('puertoFueraDeRango') && formRed.get('
        ↪ puertoServicio')?.touched) {
                        <small class="error-badge">
                            Puerto inválido: debe estar comprendido entre 1024 y 65535.
                        </small>
                    }
                </div>
                <button type="submit" [disabled]="formRed.invalid">Aplicar Conexión</button>
            </form>
        </div>
    `,
    styles: [`
        .config-box { max-width: 380px; padding: 16px; border: 1px solid #CBD5E1; border-radius: 6px
        ↪ ; background: #FFFFFF; font-family: 'Noto Sans', sans-serif; }
        h4 { color: #1F2758; margin-top: 0; }
        .campo { display: flex; flex-direction: column; gap: 4px; margin-bottom: 12px; }
        input { padding: 6px 10px; border: 1px solid #CBD5E1; border-radius: 4px; }
        .error-badge { color: #DC2626; font-size: 11px; font-weight: bold; }
        button { background: #1E75B8; color: #fff; border: none; padding: 8px 12px; border-radius: 4
        ↪ px; cursor: pointer; width: 100%; font-weight: bold; }
        button:disabled { background: #CBD5E1; cursor: not-allowed; }
    `]
})
export class ConfiguracionRedComponent implements OnInit {
    formRed!: FormGroup;

    ngOnInit(): void {
        this.formRed = new FormGroup({
            puertoServicio: new FormControl(8080, [Validators.required, puertoValido])
        });
    }
}

```

```
guardarParametros(): void {  
  if (this.formRed.valid) {  
    console.log('Parámetros de red aceptados:', this.formRed.value);  
  }  
}  
}
```

Listing 5: p3-ej/validador-puerto.ts

Explicación

- **Carga Útil Informativa en ValidationErrors:** Al devolver un objeto descriptivo con `minimoPermitido` y `maximoPermitido`, permites a la vista HTML proyectar mensajes de ayuda dinámicos y precisos al operador.
- **Inspección Atómica con `hasError()`:** El método `hasError('puertoFueraDeRango')` permite consultar específicamente la infracción sin tener que inspeccionar manualmente objetos de error complejos.
- **Composición Limpia:** El validador se integra de manera homogénea en la matriz de validaciones junto a `Validators.required`.

4 El Cliente Angular HttpClient

Para interactuar con servidores remotos de forma óptima y moderna, Angular incorpora el cliente HTTP (`HttpClient`), el cual abstrae la complejidad de las transferencias de red bajo el paradigma reactivo utilizando flujos asíncronos gobernados por Observables de RxJS (ver ANEXO III).

4.1 Inyección del Servicio y Configuración Standalone

En la arquitectura autónoma y libre de módulos de Angular 19, habilitas el soporte de peticiones HTTP registrando la función proveedora `provideHttpClient()` en el archivo de configuración global `src/app/app.config.ts`. Posteriormente, inyectas el servicio dentro de tus clases mediante la función funcional `inject(HttpClient)`:

```
import { ApplicationConfig } from '@angular/core';  
import { provideHttpClient } from '@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient()  
  ]  
};
```

Listing 6: p4-http/src/app/app.config.ts

4.2 Scaffolding de Servicios y Arquitectura de Operaciones CRUD

Un estándar de ingeniería riguroso dicta que los componentes de la interfaz no deben invocar directamente las operaciones de red; en su lugar, delegan esta responsabilidad en un servicio inyectable dedicado que implemente el patrón Repositorio.

Para generar este servicio mediante Angular CLI, ejecuta en tu terminal:

```
ng generate service services/usuario
# Comando abreviado equivalente:
ng g s services/usuario
```

Angular CLI creará dos archivos en la carpeta `src/app/services/`:

- `usuario.service.ts`: Contiene la clase de servicio decorada con `@Injectable({providedIn: 'root'})`, lo que garantiza que actuará como un Singleton accesible desde cualquier componente sin necesidad de matricularlo en módulos.
- `usuario.service.spec.ts`: Esqueleto para pruebas unitarias automatizadas del servicio.

A continuación, implementa las operaciones CRUD completas en `src/app/services/usuario.service.ts`:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Usuario {
  id?: number | string;
  nombre: string;
  correo: string;
}

@Injectable({
  providedIn: 'root'
})
export class UsuarioService {
  private http = inject(HttpClient);
  private apiUrl = 'http://localhost:3000/usuarios';

  // 1. Lectura global (GET)
  obtenerTodos(): Observable<Usuario[]> {
    return this.http.get<Usuario[]>(this.apiUrl);
  }

  // 2. Creación de registro (POST)
  crear(usuario: Usuario): Observable<Usuario> {
    return this.http.post<Usuario>(this.apiUrl, usuario);
  }

  // 3. Actualización total (PUT)
  actualizar(id: number | string, usuario: Usuario): Observable<Usuario> {
    return this.http.put<Usuario>(`${this.apiUrl}/${id}`, usuario);
  }

  // 4. Eliminación física (DELETE)
  eliminar(id: number | string): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Listing 7: p4-http/usuario.service.ts

Miniejercicio: Servicio de Consulta Tipada con HttpClient

Enunciado: Desarrolla un servicio inyectable en la raíz denominado `SensorService` (ubicado en `src/app/services/sensor.service.ts`) que utilice `inject(HttpClient)` para recuperar un listado de métricas de telemetría fuertemente tipadas mediante una interfaz `Metrica`.

Solución:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Metrica {
  id: string;
  nodo: string;
  temperatura: number;
}

@Injectable({ providedIn: 'root' })
export class SensorService {
  private http = inject(HttpClient);
  private endpoint = 'https://api.empresa.org/v1/metricas';

  obtenerMetricas(): Observable<Metrica[]> {
    return this.http.get<Metrica[]>(this.endpoint);
  }
}
```

Listing 8: p4-http/p4-mini-sensor.service.ts

Explicación: Al parametrizar el método genérico `http.get<Metrica[]>`, TypeScript garantiza en tiempo de compilación que el flujo `Observable` transporte exactamente el contrato de datos esperado por los componentes.

5 Manejo de Errores Robustos con RxJS

Las comunicaciones web están expuestas a imprevistos de red como pérdidas súbitas de señal, servidores no disponibles o peticiones malformadas. Angular permite implementar flujos de contingencia altamente resistentes gracias a los operadores funcionales de RxJS, especialmente `catchError` y `retry` (ver ANEXO III).

El operador `catchError` intercepta cualquier evento de error dentro del flujo del `Observable`, dándote la oportunidad de analizar el código de estado HTTP recibido (ver ANEXO II), adoptar un mecanismo de contingencia (*fallback*) o transformar la excepción técnica en un mensaje comprensible para el usuario:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpResponseError } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';
import { Usuario } from './usuario.service';

@Injectable({ providedIn: 'root' })
export class UsuarioResilienteService {
  private http = inject(HttpClient);
  private apiUrl = 'http://localhost:3000/usuarios';
```

```

crearConSeguridad(usuario: Usuario): Observable<Usuario> {
  return this.http.post<Usuario>(this.apiUrl, usuario).pipe(
    retry(2), // Reintenta la llamada hasta 2 veces antes de bifurcar a error
    catchError(this.gestionarError)
  );
}

private gestionarError(error: HttpResponse): Observable<never> {
  let mensajeUsuario = 'Ha ocurrido un error inesperado en la comunicación.';
  if (error.status === 0) {
    mensajeUsuario = 'Imposible conectar con el servidor. Comprueba tu conexión a red.';
  } else if (error.status === 400) {
    mensajeUsuario = 'Petición incorrecta o datos inválidos en el formulario.';
  } else if (error.status === 404) {
    mensajeUsuario = 'El recurso solicitado no fue localizado en el servidor remoto.';
  } else if (error.status >= 500) {
    mensajeUsuario = 'Incidencia interna en el servidor remoto. Inténtalo más tarde.';
  }
  return throwError(() => new Error(mensajeUsuario));
}
}

```

Listing 9: p5-errores/usuario-resiliente.service.ts

Miniejercicio: Intercepción de Fallo con catchError y Fallback Seguro

Enunciado: Implementa un método de consulta en un servicio que intente descargar una lista de avisos y, en caso de caída o error del servidor, intercepte la anomalía con `catchError` retornando una lista vacía de contingencia sin romper el flujo.

Solución:

```

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, of } from 'rxjs';
import { catchError } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class AvisosService {
  private http = inject(HttpClient);

  cargarAvisosSeguros(): Observable<string[]> {
    return this.http.get<string[]>('/api/avisos-urgentes').pipe(
      catchError((error) => {
        console.warn('Fallo al recuperar avisos, aplicando fallback local:', error.status);
        return of([]); // Retorna un flujo con arreglo vacio para evitar caída de UI
      })
    );
  }
}

```

Listing 10: p5-errores/p5-mini-fallback.service.ts

Explicación: Al sustituir el flujo fallido por `of([])`, el componente visual consumidor no se rompe ni lanza errores no capturados, asegurando que la interfaz continúe funcionando de forma degradada pero estable.

6 Ejercicio Final

Sistema Integrado de Registro con Validación y Persistencia en API Mock

Enunciado Diseña e implementa una solución completa de ingeniería en Angular 19 que articule la captura, validación estricta y persistencia de operadores en una base de datos local simulada con JSON Server. La solución debe estructurarse paso a paso mediante las siguientes directivas:

1. Configurar una base de datos local simulada (db.json) y arrancarla en el puerto local 3000 con JSON Server.
2. Construir el servicio `UsuarioService` utilizando inyección funcional con `inject(HttpClient)` y tipado estricto.
3. Desarrollar mediante Angular CLI el componente standalone `AltaUsuarioComponent` articulado sobre un `FormGroup` reactivo que exija: nombre obligatorio con un mínimo de 3 caracteres y correo corporativo con formato válido.
4. Ejecutar la petición POST al enviar el formulario, gestionar la respuesta asíncrona mediante suscripción, refrescar los controles con `reset()` y notificar el resultado visual al usuario mediante avisos diferenciados de éxito y error.

Solución Fase 1: Configuración de la API Mock y arranque del servidor local

Genera el archivo `db.json` en la raíz de tu espacio de trabajo para definir el esquema de datos:

```
{
  "usuarios": [
    { "id": "1", "nombre": "Operador Sistema", "correo": "operador@empresa.org" }
  ]
}
```

Listing 11: p6-ejfinal/db.json

Inicializa el servidor simulado ejecutando el comando de terminal correspondiente:

```
npx json-server --watch db.json --port 3000
```

Fase 2: Generación del Servicio y del Componente con Angular CLI

Abre una nueva terminal en tu proyecto y ejecuta los siguientes comandos de scaffolding:

```
# 1. Generar el servicio en src/app/services/usuario.service.ts
ng g s services/usuario

# 2. Generar el componente de alta en src/app/components/alta-usuario/
ng g c components/alta-usuario --inline-template --inline-style
```

Fase 3: Implementación del Componente de Alta Integrado

Edita el archivo generado en `src/app/components/alta-usuario/alta-usuario.component.ts`:

```

import { Component, OnInit, inject } from '@angular/core';
import { FormGroup, FormBuilder, Validators, ReactiveFormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { UsuarioService, Usuario } from '../services/usuario.service';

@Component({
  selector: 'app-alta-usuario',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  template: `
    <div class="panel-registro">
      <h2>Registro de Nuevo Operador</h2>

      @if (mensajeEstado) {
        <div [class]="esError ? 'alerta alerta-error' : 'alerta alerta-exito'">
          {{ mensajeEstado }}
        </div>
      }

      <form [formGroup]="registroForm" (ngSubmit)="registrarUsuario()">
        <div class="campo">
          <label for="nombre">Nombre Completo:</label>
          <input id="nombre" type="text" formControlName="nombre" placeholder="Mínimo 3
↪ caracteres" />
          @if (registroForm.get('nombre')?.invalid && registroForm.get('nombre')?.touched) {
            <small class="error-msg">El nombre es obligatorio y debe tener al menos 3 caracteres
↪ </small>
          }
        </div>

        <div class="campo">
          <label for="correo">Correo Electrónico:</label>
          <input id="correo" type="email" formControlName="correo" placeholder="ejemplo@empresa.
↪ com" />
          @if (registroForm.get('correo')?.invalid && registroForm.get('correo')?.touched) {
            <small class="error-msg">Introduce una dirección de correo válida.</small>
          }
        </div>

        <button type="submit" [disabled]="registroForm.invalid">Guardar en Base de Datos</button>
↪ >
      </form>
    </div>
  `,
  styles: [`
    .panel-registro { max-width: 440px; margin: 20px auto; padding: 24px; border: 1px solid #
↪ CBD5E1; border-radius: 8px; font-family: 'Noto Sans', sans-serif; background: #FFFFFF; }
    h2 { color: #1F2758; margin-bottom: 16px; font-size: 1.25rem; }
    .campo { margin-bottom: 14px; display: flex; flex-direction: column; gap: 4px; }
    label { font-size: 13px; font-weight: bold; color: #2D3748; }
    input { padding: 8px 12px; border: 1px solid #CBD5E1; border-radius: 4px; font-size: 14px; }
    .error-msg { color: #DC2626; font-size: 11px; }
    button { background: #1E75B8; color: #fff; border: none; padding: 10px; border-radius: 4px;
↪ font-weight: bold; cursor: pointer; width: 100%; margin-top: 10px; }
    button:disabled { background: #CBD5E1; cursor: not-allowed; }
    .alerta { padding: 10px; border-radius: 4px; margin-bottom: 14px; font-size: 13px; font-

```

```
    ↵ weight: bold; }
    .alerta-exito { background: #F0FDF4; color: #059669; border: 1px solid #86EFAC; }
    .alerta-error { background: #FEF2F2; color: #DC2626; border: 1px solid #FCA5A5; }
  `]
})
export class AltaUsuarioComponent implements OnInit {
  private fb = inject(FormBuilder);
  private usuarioService = inject(UsuarioService);

  registroForm!: FormGroup;
  mensajeEstado = '';
  esError = false;

  ngOnInit(): void {
    this.registroForm = this.fb.group({
      nombre: ['', [Validators.required, Validators.minLength(3)]],
      correo: ['', [Validators.required, Validators.email]]
    });
  }

  registrarUsuario(): void {
    if (this.registroForm.invalid) return;

    const nuevoUsuario: Usuario = this.registroForm.value;

    this.usuarioService.crear(nuevoUsuario).subscribe({
      next: (usuarioCreado) => {
        this.esError = false;
        this.mensajeEstado = `Usuario ${usuarioCreado.nombre} registrado con éxito en el
        ↵ servidor.`;
        this.registroForm.reset();
      },
      error: (err: Error) => {
        this.esError = true;
        this.mensajeEstado = `No se pudo completar el registro: ${err.message}`;
      }
    });
  }
}
```

Listing 12: p6-ejfinal/alta-usuario.component.ts

Explicación

- **Separación Estricta de Responsabilidades:** El componente gobierna exclusivamente la presentación visual y la interacción reactiva, delegando la conexión y serialización de red al servicio `UsuarioService`.
- **Gestión Integral de Ciclo de Formulario:** Tras completar la persistencia exitosa (código 201 Created, ver ANEXO II), la invocación a `reset()` devuelve los controles a su estado original (`pristine`), previniendo transmisiones duplicadas.
- **Tolerancia a Excepciones de Red:** Los bloques `next` y `error` de la suscripción asíncrona garantizan que cualquier indisponibilidad del backend sea comunicada de manera clara al operador mediante avisos estilizados.

7 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación de los formularios reactivos y del consumo estructurado de servicios REST en Angular 19 proporciona una base de desarrollo robusta, modular y altamente eficiente para aplicaciones corporativas:

- **Arquitectura Inmutable de Formularios:** El gobierno explícito mediante `FormControl` y `FormGroup` erradica las inconsistencias de datos, ofreciendo un control absoluto sobre el valor y la validez en memoria (ver ANEXO I).
- **Validación Integral Desacoplada:** La combinación de validadores nativos con funciones de comprobación lógica personalizadas previene la persistencia de datos anómalos sin acoplarse al DOM.
- **Consumo de Red con HttpClient Moderno:** La inyección funcional mediante `provideHttpClient()` simplifica el arranque de la aplicación y permite tipar las transferencias asíncronas con interfaces de TypeScript.
- **Resiliencia Reactiva con RxJS:** La integración de operadores funcionales como `catchError` y `retry` dentro de tuberías asíncronas garantiza tolerancia activa ante incidencias de comunicación (ver ANEXO III).
- **Dominio de Estándares Web:** La comprensión rigurosa de los códigos de respuesta HTTP (ver ANEXO II) facilita la depuración metódica de los servicios de backend y mejora sensiblemente la calidad del software entregado.

ANEXO I Glosario de Términos Técnicos

1. **Reactive Forms (Formularios Reactivos):** Enfoque de gestión de formularios en Angular donde el modelo de datos y las directivas de validación se definen de manera explícita e inmutable en TypeScript, mapeándose posteriormente a la plantilla HTML.
2. **FormControl:** Entidad básica que encapsula el valor, el estado de validación y los eventos de interacción de un campo unitario de entrada.
3. **FormGroup:** Estructura jerárquica que agrupa múltiples controles (`FormControl`) u otros subgrupos, calculando su estado de validez y valor agregado de forma consolidada.
4. **FormBuilder:** Servicio auxiliar inyectable que facilita la creación sintáctica abreviada de instancias complejas de `FormGroup` y `FormControl`.
5. **Pristine / Dirty:** Metadatos de interacción. `pristine` indica que aún no has modificado el valor inicial del control; `dirty` confirma que el valor ha sufrido alteraciones manuales en la vista.
6. **Untouched / Touched:** Propiedades de foco visual. Un campo es clasificado como `touched` una vez que ha recibido y perdido el foco del cursor (*blur*).
7. **Custom Validator (Validador Personalizado):** Función pura que extiende el sistema de validaciones evaluando un `AbstractControl` y retornando un mapa de errores (`ValidationErrors`) o `null` si es válido.
8. **HttpClient:** Módulo cliente de Angular que abstrae y optimiza el protocolo HTTP sobre flujos reactivos de RxJS para ejecutar operaciones asíncronas tipadas en red.
9. **provideHttpClient():** Función de configuración funcional utilizada en proyectos autónomos de Angular 19 para habilitar los servicios de red sin requerir módulos de infraestructura legados.
10. **catchError:** Operador funcional de RxJS que intercepta errores producidos dentro de una tubería de ejecución reactiva para aplicar acciones de recuperación o relanzar excepciones controladas.
11. **API Mock:** Servidor de pruebas local que simula el comportamiento y endpoints de una API REST productiva consumiendo un archivo de datos plano (por ejemplo, mediante JSON Server).
12. **Idempotencia:** Propiedad de los métodos HTTP (como GET, PUT y DELETE) por la cual múltiples peticiones idénticas consecutivas producen exactamente el mismo efecto sobre el estado de persistencia en el servidor.

ANEXO II Manual de Códigos de Estado HTTP en APIs REST

Guía Didáctica de Referencia para el Desarrollo y Depuración

1 Introducción y Arquitectura de Comunicación HTTP

En el desarrollo de servicios web modernos y arquitecturas basadas en APIs (Application Programming Interfaces), la comunicación asertiva entre el cliente (frontend web, aplicación móvil o servicios externos) y el servidor es fundamental. El protocolo HTTP (Hypertext Transfer Protocol) define un estándar global para dicha comunicación a través de los Códigos de Estado HTTP.

Un código de estado es un número de tres dígitos devuelto por el servidor como respuesta a una petición realizada por el cliente. Estos códigos permiten estructurar la lógica de negocio del lado del frontend, indicando de forma inmediata si la operación concluyó con éxito, si requiere redirección o si ocurrió un error en alguno de los extremos de la comunicación. Comprender la semántica exacta de cada código es la diferencia entre una API robusta y una propensa a fallos de integración.

2 Bloque 2xx: Respuestas Exitosas

La serie 2xx indica que la petición del cliente fue recibida, entendida y aceptada correctamente por el servidor. Son los códigos de retorno habituales en un flujo normal de operación:

- **200 OK:** Código estándar por excelencia para indicar éxito. Significa que la petición se procesó correctamente y la respuesta contiene el cuerpo con la información solicitada (frecuente en consultas GET, o actualizaciones PUT/PATCH que retornan el objeto modificado).
- **201 Created:** Confirma que la solicitud ha culminado exitosamente creando un nuevo recurso en el servidor (típico de peticiones POST). Es una buena práctica que incluya la cabecera `Location` con la URI del nuevo elemento.
- **202 Accepted:** La petición ha sido aceptada para su procesamiento, pero este no ha finalizado o ni siquiera ha comenzado. Es de uso común en procesamientos asíncronos pesados y arquitecturas orientadas a eventos para evitar bloqueos de red.
- **204 No Content:** La petición se completó con éxito, pero el servidor no necesita devolver ningún contenido en el cuerpo de la respuesta (muy utilizado en operaciones de eliminación DELETE).

3 Bloque 3xx: Redirecciones

Los códigos de la serie 3xx comunican que el cliente debe realizar acciones adicionales (comúnmente redirigir su petición a otra URL) para completar con éxito la solicitud:

- **301 Moved Permanently:** El recurso solicitado ha sido reubicado de forma definitiva bajo una nueva URI. Las futuras peticiones deben utilizar esta nueva dirección.
- **302 Found (Redirección Temporal):** El recurso reside temporalmente en una URI diferente, por lo que el cliente debe seguir utilizando la URI original en llamadas sucesivas.
- **304 Not Modified:** Indica que el recurso no ha sufrido modificaciones desde la última vez que fue descargado. Se apoya en cabeceras de optimización como `ETag` o `If-Modified-`

Since, permitiendo al cliente renderizar su copia local en caché y reduciendo el consumo de ancho de banda.

4 Bloque 4xx: Errores del Cliente

Esta familia de códigos indica que la anomalía ha sido provocada por el emisor de la petición (el cliente frontend), ya sea por sintaxis incorrecta, falta de credenciales o por consultar recursos no existentes:

- **400 Bad Request:** El servidor no puede procesar la solicitud debido a un fallo percibido en el cliente (sintaxis inválida, formato JSON corrupto o parámetros ausentes).
- **401 Unauthorized:** La petición requiere autenticación previa del usuario. El cliente debe autenticarse aportando credenciales o un token válido (ej. `Authorization: Bearer <token>`).
- **403 Forbidden:** El cliente aportó credenciales válidas (el servidor reconoce su identidad), pero carece de los privilegios de rol necesarios para interactuar con ese recurso concreto.
- **404 Not Found:** El servidor no encuentra el recurso solicitado o el endpoint no existe en la cartografía de rutas.
- **405 Method Not Allowed:** El método HTTP empleado (ej. POST) no está habilitado para el endpoint invocado.
- **409 Conflict:** La petición choca con el estado actual del servidor (por ejemplo, registrar un usuario con un correo que ya existe en la base de datos).
- **422 Unprocessable Entity:** La sintaxis de la petición es correcta, pero contiene errores semánticos de negocio que impiden su procesamiento.
- **429 Too Many Requests:** El cliente ha emitido demasiadas peticiones en una ventana de tiempo acotada (*rate limiting*), protegiendo el sistema contra abusos o ataques DoS.

5 Bloque 5xx: Errores del Servidor

Los códigos de la serie 5xx indican situaciones en las que el servidor es consciente de haber fallado o de encontrarse indisponible para resolver la petición debido a problemas internos:

- **500 Internal Server Error:** Código genérico para reportar un fallo no controlado del lado del backend. En entornos productivos, jamás debe filtrarse el *stack trace* en el payload por razones de seguridad.
- **501 Not Implemented:** El servidor carece de la funcionalidad o método necesario para responder a la solicitud.
- **502 Bad Gateway:** Un servidor actuando como proxy o pasarela recibió una respuesta inválida del servidor ascendente al que intentó conectarse.
- **503 Service Unavailable:** El servidor no está disponible en este momento debido a sobrecarga de tráfico o tareas programadas de mantenimiento.
- **504 Gateway Timeout:** El servidor actuando como puerta de enlace no recibió una respuesta a tiempo de una base de datos o servicio secundario.

Miniejercicio: Clasificador Semántico de Respuestas HTTP

Enunciado: Diseña una función utilitaria TypeScript que reciba un código numérico HTTP y devuelva una cadena descriptiva para el operador de la aplicación.

Solución:

```
export function categorizarCodigoHttp(codigo: number): string {
  if (codigo >= 200 && codigo < 300) return 'Operación completada con éxito';
  if (codigo === 401 || codigo === 403) return 'Bloqueo por permisos o sesión inválida';
  if (codigo === 404) return 'Recurso no localizado en el servidor';
  if (codigo >= 500) return 'Incidencia técnica en el servidor remoto';
  return 'Código de respuesta fuera de categoría estándar';
}
```

Listing 13: anx2-p5-mini-http-status.ts

Explicación: La discriminación controlada mediante rangos numéricos permite mapear los códigos HTTP a notificaciones comprensibles en la capa visual de tu aplicación.

ANEXO III Observables en Angular v19

Guía de Arquitectura, Interoperabilidad y Patrones Reactivos

1 El Rol Esencial de los Observables en la Era de los Signals

Con la consolidación de la API de Signals en Angular 19 como el mecanismo predeterminado para el control de estado síncrono y la renderización granular en la interfaz de usuario, ha surgido un debate recurrente en los equipos de desarrollo: ¿Siguen siendo necesarios los Observables?

La respuesta técnica de la arquitectura oficial de Angular es rotunda: Sí. Los Signals y los Observables no compiten entre sí, sino que se complementan de manera simbiótica cubriendo dimensiones reactivas distintas:

- **Signals (Reactividad de Estado Síncrono):** Diseñados para representar valores que varían a lo largo del tiempo y cuyo valor presente siempre está disponible de manera síncrona. Son idóneos para enlazar propiedades a la plantilla HTML, ya que carecen del concepto de completado o error y eliminan por completo la necesidad de suscripciones manuales.
- **Observables (Reactividad de Eventos y Asincronía Compleja):** Diseñados para coordinar secuencias de eventos asíncronos distribuidos en el tiempo. Resultan indispensables para modelar flujos continuos, cancelaciones explícitas de peticiones, reintentos automatizados, demoras temporales, gestión de errores en red y composición de flujos concurrentes.

Cuadro 1: Matriz de selección reactiva en Angular 19

Requerimiento Arquitectónico	Signals	Observables (RxJS)
Lectura síncrona del valor actual	Sí (invocación directa)	No (requiere suscripción o pipe)
Enlace directo con la plantilla HTML	Óptimo (reactividad fina $\mathcal{O}(k)$)	Requiere AsyncPipe
Operaciones asíncronas de red (HTTP)	No recomendado	Óptimo (HttpClient)
Gestión de tiempo (debounce, delay)	No soportado nativamente	Excelente (operadores funcionales)
Composición de flujos en paralelo	Complejo	Excelente (forkJoin, combineLatest)
Canales bidireccionales (WebSockets)	No soportado	Óptimo (WebSocketSubject)

2 Interoperabilidad de RxJS con la API de Signals

El paquete oficial `@angular/core/rxjs-interop` provee los puentes de comunicación necesarios para combinar ambos mundos sin fricciones, permitiendo aprovechar la potencia asíncrona de RxJS en los servicios con la ergonomía de Signals en los componentes.

2.1 La función `toSignal`

Permite suscribirse a un Observable y exponer sus emisiones como un Signal de solo lectura. Angular se encarga automáticamente de desuscribirse del flujo original cuando el contexto (componente o servicio inyectable) es destruido:

```
import { inject, Component } from '@angular/core';
```

```

import { HttpClient } from '@angular/common/http';
import { toSignal } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-users',
  standalone: true,
  template: `
    <ul>
      @for (user of users(); track user.id) {
        <li>{{ user.name }}</li>
      }
    </ul>
  `
})
export class UsersComponent {
  private http = inject(HttpClient);

  // toSignal requiere un valor inicial si el Observable no emite sincronamente
  users = toSignal(
    this.http.get<any[]>('/api/users'),
    { initialValue: [] }
  );
}

```

Listing 14: anx3-p2-to-signal.component.ts

2.2 La función toObservable

Permite convertir un Signal en un flujo Observable. Esto es de vital importancia cuando desees reaccionar a las variaciones de una señal síncrona para desencadenar efectos asíncronos de red (como ejecutar una búsqueda tras alterar un término de filtrado):

```

import { Component, signal, inject } from '@angular/core';
import { toObservable } from '@angular/core/rxjs-interop';
import { HttpClient } from '@angular/common/http';
import { switchMap } from 'rxjs';

@Component({
  selector: 'app-product-details',
  standalone: true,
  template: `<h3>Detalles del producto ID: {{ productId() }}</h3>`
})
export class ProductDetailsComponent {
  private http = inject(HttpClient);

  productId = signal<number>(1);

  // Convertimos a Observable para enganchar el switchMap reactivo de red
  product$ = toObservable(this.productId).pipe(
    switchMap(id => this.http.get(`/api/products/${id}`))
  );
}

```

Listing 15: anx3-p2-to-observable.component.ts

Miniejercicio: Sincronización Reactiva con toSignal

Enunciado: Desarrolla un componente que consuma un flujo numérico periódico (`interval(1000)`) y lo exponga en la plantilla como una señal síncrona mediante `toSignal`, estableciendo un valor inicial en cero.

Solución:

```
import { Component } from '@angular/core';
import { interval } from 'rxjs';
import { toSignal } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-contador-stream',
  standalone: true,
  template: `<p>Contador en tiempo real: <strong>{{ ticks() }}</strong></p>`
})
export class ContadorStreamComponent {
  ticks = toSignal(interval(1000), { initialValue: 0 });
}
```

Listing 16: `anx3-p2-mini-to-signal.component.ts`

Explicación: La función `toSignal` resuelve automáticamente la suscripción al Observable `interval` y cancela la escucha al desmontarse el componente sin necesidad de ganchos de ciclo de vida manuales.

3 Casos de Uso Críticos en el Desarrollo Moderno

A continuación se detallan escenarios avanzados donde el uso de Observables resulta insustituible frente a una solución puramente basada en Signals.

3.1 Búsqueda Interactiva en Tiempo Real (Typeahead)

Al redactar términos en un control de búsqueda, es imprescindible regular la tasa de peticiones transmitidas al servidor y cancelar consultas en vuelo que hayan quedado obsoletas:

```
import { Component, signal, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { toObservable, toSignal } from '@angular/core/rxjs-interop';
import { debounceTime, distinctUntilChanged, switchMap, of, catchError } from 'rxjs';

@Component({
  selector: 'app-search',
  standalone: true,
  template: `
    <input (input)="onInputChange($any($event.target).value)" placeholder="Buscar elementos..." />
    <ul>
      @for (item of results(); track item.id) {
        <li>{{ item.name }}</li>
      }
    </ul>
  `
})
export class SearchComponent {
  private http = inject(HttpClient);
  query = signal<string>('');
}
```

```

results = toSignal(
  toObservable(this.query).pipe(
    debounceTime(400),          // Regula rafagas rapidas de escritura
    distinctUntilChanged(),    // Emite unicamente si el texto cambio
    switchMap(term => {
      if (!term.trim()) return of([]);
      return this.http.get<any[]>(`/api/search?q=${term}`).pipe(
        catchError(() => of([])) // Previene la caida del flujo ante errores
      );
    })
  ),
  { initialValue: [] }
);

onInputChange(val: string) {
  this.query.set(val);
}
}

```

Listing 17: anx3-p3-typeahead.component.ts

3.2 Consumo de Canales Bidireccionales con WebSockets

Las conexiones que transmiten flujos continuos mediante WebSockets no se pueden modelar directamente mediante Signals debido a la naturaleza bidireccional y abierta del protocolo:

```

import { Injectable } from '@angular/core';
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class TelemetryService {
  private socket$: WebSocketSubject<any> = websocket('wss://telemetry.empresa.org');

  getStream(): Observable<any> {
    return this.socket$.asObservable();
  }

  sendAction(action: string, payload: any): void {
    this.socket$.next({ action, payload });
  }
}

```

Listing 18: anx3-p3-telemetry.service.ts

Miniejercicio: Amortiguación de Búsqueda con debounceTime

Enunciado: Construye una tubería reactiva sobre una señal de término de filtrado que descarte valores idénticos consecutivos y retrase la emisión 300 ms antes de notificar en consola.

Solución:

```

import { Component, signal } from '@angular/core';
import { toObservable } from '@angular/core/rxjs-interop';
import { debounceTime, distinctUntilChanged } from 'rxjs';

@Component({

```

```

    selector: 'app-filtro-controlado',
    standalone: true,
    template: `<input (input)="filtro.set($any($event.target).value)" placeholder="Filtrar..." />`
  })
  export class FiltroControladoComponent {
    filtro = signal<string>('');

    constructor() {
      toObservable(this.filtro).pipe(
        debounceTime(300),
        distinctUntilChanged()
      ).subscribe(valor => console.log('Filtro estabilizado:', valor));
    }
  }
}

```

Listing 19: anx3-p3-mini-debounce.component.ts

Explicación: `debounceTime(300)` frena emisiones aceleradas por tecleo rápido, mientras que `distinctUntilChanged()` suprime eventos repetidos si el valor final no ha variado.

4 Gestión del Ciclo de Vida y Prevención de Fugas de Memoria

Uno de los mayores desafíos al operar con Observables es asegurar la liberación de suscripciones activas para evitar retenciones accidentales en memoria (*Memory Leaks*). En Angular 19, el operador `takeUntilDestroyed()` resuelve este problema sin requerir la implementación imperativa de `ngOnDestroy`.

4.1 Uso Implícito en Contexto de Inyección

Si el operador se declara dentro del contexto de inyección (durante la inicialización de propiedades o en el constructor), detecta automáticamente el `DestroyRef` de Angular:

```

import { Component, OnInit } from '@angular/core';
import { interval } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-auto-logger',
  standalone: true,
  template: `<p>Monitor de trazas activo con desuscripción automática.</p>`
})
export class AutoLoggerComponent implements OnInit {
  private heartbeat$ = interval(1000).pipe(
    takeUntilDestroyed() // Auto-configuración contextual
  );

  ngOnInit() {
    this.heartbeat$.subscribe(val => console.log('Heartbeat emitido:', val));
  }
}

```

Listing 20: anx3-p4-auto-logger.component.ts

4.2 Uso Fuera del Contexto de Inyección

Si necesitas vincular el operador en métodos tardíos de ciclo de vida (como `ngOnInit`), inyectas

explícitamente DestroyRef y lo suministras como parámetro:

```
import { Component, inject, OnInit, DestroyRef } from '@angular/core';
import { interval } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-dynamic-timer',
  standalone: true,
  template: `<p>Temporizador dinámico protegido.</p>`
})
export class DynamicTimerComponent implements OnInit {
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    interval(3000).pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(tick => console.log('Tick seguro fuera de inyección:', tick));
  }
}
```

Listing 21: anx3-p4-dynamic-timer.component.ts

Miniejercicio: Blindaje contra Fugas de Memoria con takeUntilDestroyed

Enunciado: Desarrolla un componente que inicie una suscripción asíncrona a un evento de reloj en el gancho ngOnInit utilizando DestroyRef explícito para asegurar su desuscripción al destruirse.

Solución:

```
import { Component, OnInit, inject, DestroyRef } from '@angular/core';
import { interval } from 'rxjs';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-reloj-protegido',
  standalone: true,
  template: `<p>Temporizador seguro en ejecución.</p>`
})
export class RelojProtegidoComponent implements OnInit {
  private destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    interval(1000).pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(s => console.log('Segundo transcurrido:', s));
  }
}
```

Listing 22: anx3-p4-mini-leak-guard.component.ts

Explicación: Al suministrar destroyRef a takeUntilDestroyed, la suscripción creada en ngOnInit queda vinculada al ciclo de vida del componente, cancelándose inmediatamente al removerse del DOM.